Customer Offer Engagement – Technical Document

Project: Customer Offer Engagement Dashboard (Power BI)

Prepared by: Meghna Garg [CPDA Batch 11]

1. Data sources

File	Rows	Content
customers.csv	17,000	One row per customer: customer_id, gender, age, income, became_member_on
events.csv	306,534	Offer received / viewed / completed and transaction events; details stored as JSON text in "value"; time in hours
offers.csv	10	Offer attributes: offer_id, offer_type, difficulty, reward, duration, channels
data_dictionary.csv	18	Table, field, Description

All files are loaded with Get Data → Text/CSV, File Origin 65001 (UTF-8), and transformed in Power Query before loading.

2. Data issues and cleaning steps

Every issue found during profiling (Column quality / distribution, based on the entire data set) and the action taken:

#	Table	Column	Issue found	Action taken	Result
1	All CSVs	(file)	Import defaulted to File Origin 1252	Set File Origin to 65001: UTF-8	Standard encoding; no values changed
2	Customers	became_member_on	Stored as number yyyymmdd; Number→Date gave errors	Changed type Text → Date; renamed member_date	0% errors on entire data set
3	Customers	age	118 used as placeholder for unknown age	Replaced 118 with null	Avg age 54.4 (would be 62.5 with 118 left in)
4	Customers	gender	Blanks loaded as empty text (not null)	Replaced empty with "Unknown"; added gender_label (Male/Female/Other/Unknown)	2,175 Unknown kept as own segment
5	Customers	income	Missing for the same 2,175 customers	Left as null (no basis to estimate)	Nulls skipped in averages
6	Customers	age_group, income_bracket, member_year (new)	Segments needed for cohort analysis	Conditional Columns (null checked first); year from member_date; "65" output replaced with "65+"; types set	6 age groups, 5 income brackets, cohorts 2013–2018
7	Offers	offer_name (new)	offer_id is a 32-character code	Merge Columns: type-difficulty-reward-duration, UPPERCASE	Readable names, e.g. BOGO-10-10-7; 10 unique
8	OfferChannels (new)	channels	Several channels in one cell, e.g. ['web', 'email']	Reference of Offers; removed brackets/quotes; Split into Rows; Trim	33 rows, one per offer per channel
9	Events	value	JSON text uses single quotes (invalid JSON)	Replace Values ' → "	Valid JSON on all 306,534 rows
10	Events	value	Offer ID / amount / reward hidden in one text column	Transform → Parse JSON; sorted time descending so the expand dialog saw all keys; expanded 4 fields; sorted back	0% errors
11	Events	offer_id_extracted (new)	Key spelled "offer id" and "offer_id" in different events	Conditional Column joining both; type Text; source columns removed	100% valid on 167,581 offer rows
12	Events	amount, reward	Unclear names, type Any	Renamed transaction_amount (Decimal), reward_earned (Whole)	Correct sums and averages
13	Events	day, event_datetime (new)	time is hours since the customer joined, not a timestamp	day = time ÷ 24 + 1; merged member_date; DateTime.From([member_date]) + #duration(0,[time],0,0)	Real timestamp for every event
14	Events → Offers	offer_id	Brief asks to validate offer IDs	Left Anti merge Events vs Offers (check query, then deleted)	0 unmatched offer IDs

#	Table	Column	Issue found	Action taken	Result
15	DataDictionary	headers	Columns named Column1–3; 3 table-level rows	Use First Row as Headers; filtered blank Field	15 field rows kept

3. Power Query – JSON extraction (Events query)

The "value" column holds text such as {'offer id': '9b98…'} or {'amount': 0.83}. It was converted into columns with these Applied Steps:

1. **Replaced Value** – single quotes replaced with double quotes so the text is valid JSON.
2. **Parsed JSON** – Transform → Parse → JSON turns each cell into a Record.
3. **Sorted Rows** – time descending, because the Expand dialog only scans the first 1,000 rows (all "offer received"), which hid the other keys.
4. **Expanded value** – offer id, offer_id, amount, reward.
5. **Sorted Rows1** – time ascending again.
6. **Added Conditional Column** – offer_id_extracted = "offer id" if not null, otherwise "offer_id". The two source columns are then removed.

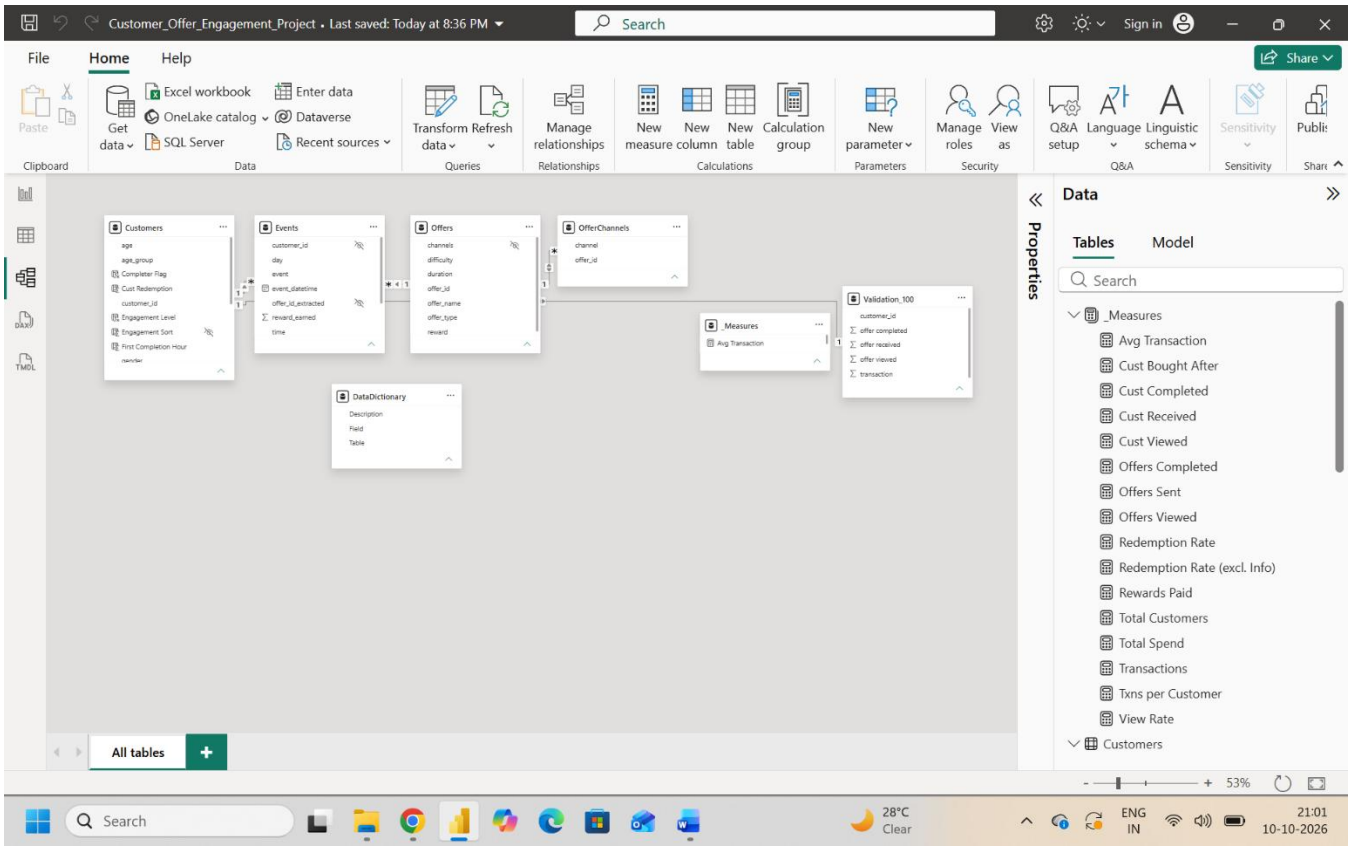
M code (from Home → Advanced Editor, Events query):

```
let
    Source = Csv.Document(File.Contents("C:\Users\meghn\Downloads\Customer_Offer_Engagement_Capstone_Project\events.csv"),[Delimiter=",", Columns=4, Encoding=65001, QuoteStyle=QuoteStyle.None]),
    #"Promoted Headers" = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    #"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers",{{"customer_id", type text}, {"event", type text}, {"value", type text}, {"time", Int64.Type}}),
    #"Replaced Value" = Table.ReplaceValue(#"Changed Type","'","""",Replacer.ReplaceText,{"value"}),
    #"Parsed JSON" = Table.TransformColumns(#"Replaced Value",{{"value", Json.Document}}),
    #"Sorted Rows" = Table.Sort(#"Parsed JSON",{{"time", Order.Descending}}),
    #"Expanded value" = Table.ExpandRecordColumn(#"Sorted Rows", "value", {"amount", "offer_id", "reward", "offer id"}, {"amount", "offer_id", "reward", "offer id"}),
    #"Sorted Rows1" = Table.Sort(#"Expanded value",{{"time", Order.Ascending}}),
    #"Added Conditional Column" = Table.AddColumn(#"Sorted Rows1", "offer_id_extracted", each if [offer id] <> null then [offer id] else [offer_id]),
    #"Changed Type1" = Table.TransformColumnTypes(#"Added Conditional Column",{{"offer_id_extracted", type text}}),
    #"Removed Columns offer id , offer_id" = Table.RemoveColumns(#"Changed Type1",{"offer_id", "offer id"}),
    #"Renamed Columns amount to transaction amount" = Table.RenameColumns(#"Removed Columns offer id , offer_id",{{"amount", "transaction_amount"}}),
    #"Changed Type2" = Table.TransformColumnTypes(#"Renamed Columns amount to transaction amount",{{"transaction_amount", type number}}),
    #"Renamed Columns1 reward to reward earned" = Table.RenameColumns(#"Changed Type2",{{"reward", "reward_earned"}}),
    #"Changed Type3" = Table.TransformColumnTypes(#"Renamed Columns1 reward to reward earned",{{"reward_earned", Int64.Type}}),
    #"Inserted Integer-Division" = Table.AddColumn(#"Changed Type3", "Integer-Division", each Number.IntegerDivide([time], 24), Int64.Type),
    #"Added to Column" = Table.TransformColumns(#"Inserted Integer-Division", {{"Integer-Division", each _ + 1, type number}}),
    #"Renamed Columns" = Table.RenameColumns(#"Added to Column",{{"Integer-Division", "day"}}),
    #"Changed Type4" = Table.TransformColumnTypes(#"Renamed Columns",{{"day", Int64.Type}}),
    #"Merged Queries" = Table.NestedJoin(#"Changed Type4", {"customer_id"}, Customers, {"customer_id"}, "Customers", JoinKind.LeftOuter),
    #"Expanded Customers" = Table.ExpandTableColumn(#"Merged Queries", "Customers", {"member_date"}, {"member_date"}),
    #"Added Custom" = Table.AddColumn(#"Expanded Customers", "event_datetime", each DateTime.From([member_date]) + #duration(0, [time], 0, 0)),
    #"Changed Type5" = Table.TransformColumnTypes(#"Added Custom",{{"event_datetime", type datetime}}),
    #"Removed Columns" = Table.RemoveColumns(#"Changed Type5",{"member_date"})
```

```
in
    #"Removed Columns"
```

4. Data model (star schema)

Events is the central fact table (one row per event, with event_datetime, transaction_amount and reward_earned). Customers and Offers are dimension tables; OfferChannels is a bridge for channel analysis. DataDictionary is a stand-alone reference table.



From (many)	To (one)	Cardinality	Cross-filter	Notes
Events[customer_id]	Customers[customer_id]	Many-to-one	Single	Auto-detected, checked
Events[offer_id_extracted]	Offers[offer_id]	Many-to-one	Single	Created manually (column names differ)
OfferChannels[offer_id]	Offers[offer_id]	Many-to-one	Both	Lets a channel filter reach Events

Technical columns (customer_id, offer_id, channels, sort columns) are hidden from report view; numeric ID-like fields are set to "Don't summarize". Engagement Level and Txn Period are sorted by their Sort columns.

5. DAX measures

Measure	DAX	What it shows
Total Customers	`COUNTROWS(Customers)`	All customers (17,000)
Offers Sent	`CALCULATE(COUNTROWS(Events), Events[event] = "offer received")`	Offers received
Offers Viewed	`CALCULATE(COUNTROWS(Events), Events[event] = "offer viewed")`	Offers viewed
Offers Completed	`CALCULATE(COUNTROWS(Events), Events[event] = "offer completed")`	Offers completed

Measure	DAX	What it shows
Transactions	`CALCULATE(COUNTROWS(Events), Events[event] = "transaction")`	Number of purchases
Total Spend	`SUM(Events[transaction_amount])`	Total money spent
Avg Transaction	`AVERAGE(Events[transaction_amount])`	Average purchase value
Rewards Paid	`SUM(Events[reward_earned])`	Reward cost of completed offers
View Rate	`DIVIDE([Offers Viewed], [Offers Sent])`	% of sent offers viewed
Redemption Rate	`DIVIDE([Offers Completed], [Offers Sent])`	% of sent offers completed
Redemption Rate (excl. Info)	`DIVIDE([Offers Completed], CALCULATE([Offers Sent], Offers[offer_type] <> "informational"))`	Redemption on offers that can be completed
Txns per Customer	`DIVIDE([Transactions], CALCULATE(DISTINCTCOUNT(Events[customer_id]), Events[event] = "transaction"))`	Purchase frequency per buying customer
Cust Received / Viewed / Completed	`CALCULATE(DISTINCTCOUNT(Events[customer_id]), Events[event] = "offer received")` (same for viewed / completed)	Funnel stages (unique customers)
Cust Bought After	`CALCULATE(DISTINCTCOUNT(Events[customer_id]), Events[Txn Period] = "After")`	Funnel last stage

Calculated columns

Table[Column]	DAX	Purpose
Customers[Offers Received]	`COUNTROWS(FILTER(RELATEDTABLE(Events), Events[event] = "offer received"))`	Offers per customer
Customers[Offers Done]	`COUNTROWS(FILTER(RELATEDTABLE(Events), Events[event] = "offer completed"))`	Completions per customer
Customers[Cust Redemption]	`DIVIDE([Offers Done], [Offers Received])`	Customer redemption rate
Customers[Engagement Level]	`SWITCH(TRUE(), [Offers Received] = 0, "No Offers", [Cust Redemption] >= 0.6, "High", [Cust Redemption] >= 0.3, "Medium", "Low")`	Engagement cohort
Customers[Engagement Sort]	`SWITCH(TRUE(), [Offers Received] = 0, 4, [Cust Redemption] >= 0.6, 1, [Cust Redemption] >= 0.3, 2, 3)`	Sort order for Engagement Level
Customers[Completer Flag]	`IF([Offers Done] > 0, "Completed an offer", "Never completed")`	Completer vs non-completer
Customers[First Completion Hour]	`MINX(FILTER(RELATEDTABLE(Events), Events[event] = "offer completed"), Events[time])`	When the first offer was completed
Events[Txn Period]	`VAR fc = RELATED(Customers[First Completion Hour])` `RETURN SWITCH(TRUE(), Events[event] <> "transaction", BLANK(), ISBLANK(fc), "No completion", Events[time] < fc, "Before", Events[time] = fc, "At completion", "After")`	Before / after first completion
Events[Txn Sort]	Same logic as Txn Period, returning 1–5	Sort order for Txn Period

6. Report pages

- **1 Overview** – KPI cards (customers, offers sent, completed, redemption rate, total spend, avg transaction), trend of transactions by day.
- **2 Offer Funnel** – funnel of unique customers: 16,994 received → 16,834 viewed → 12,774 completed → 11,945 bought after.
- **3 Customer Segmentation** – redemption by gender, age group, income bracket, member year; age × income heatmap; engagement levels.
- **4 Offer Performance** – redemption by offer type, reward, channel and offer name; difficulty vs completion scatter.
- **5 Transaction Behavior** – completers vs non-completers; spending before vs after first completion.

Slicers (Gender, Age group, Income bracket, Offer type) are synced across all pages (View → Sync slicers).

7. Testing and validation

Test	How it was done	Result
DAX vs Power Query (100 customers)	Table visual of 100 sample customers with DAX measures, compared with the same customers counted in Power Query (Group By)	Totals match: 444 received / 342 viewed / 198 completed / 821 transactions
JSON parsing complete	Quality bar on offer_id_extracted for non-transaction rows (entire data set)	100% valid, 0 missing IDs on 167,581 rows
JSON vs event type	amount filled only on transactions; offer_id only on completions	45% / 11% of rows, as expected
Offer ID validation	Left Anti merge Events → Offers	0 unmatched rows
Row counts after load	Compared with source CSVs	Customers 17,000; Events 306,534; Offers 10; OfferChannels 33
Cardinality reduced	Channels split into OfferChannels; offer_name replaces 32-char ID on charts; unused columns hidden	Smaller, cleaner model
Refresh time	Home → Refresh, timed with a stopwatch	[__ seconds] (target under 3 minutes)

8. How to add new offers

1. Open offers.csv and add a new row with the same columns: offer_id (unique), offer_type (bogo / discount / informational), difficulty, reward, duration, and channels in the same format, e.g. ['web', 'email'].
2. Save the file with the same name in the same folder.
3. In Power BI, click Home → Refresh. offer_name and the OfferChannels rows are rebuilt automatically by the existing Power Query steps.
4. When events for the new offer arrive in events.csv, they link to it through offer_id_extracted → Offers[offer_id].
5. Check: repeat the Left Anti merge (Events vs Offers) – it should return 0 rows. If a new channel name appears, it shows automatically in channel charts.

New customers or events are handled the same way: replace the CSV and refresh. No DAX changes are needed.

9. Publishing and refresh

- Publish: Home → Publish to a workspace in Power BI Service.
- Permissions: workspace roles (Admin / Member / Viewer) or share the report with view-only access.
- Scheduled refresh: dataset Settings → Data source credentials (an On-premises data gateway is needed for local CSV files) → Scheduled refresh, e.g. daily at 6:00 AM.

Not published as don't have school or work account

10. Key findings and recommendations

Area	Finding
Overall	76,277 offers sent; 75.7% viewed; 44.0% completed (55.0% excluding informational offers)
Offer type	Discount 58.6% redemption vs BOGO 51.4%; best single offer DISCOUNT-10-2-10 at 70.0%
Reward	Smaller rewards redeem more: $2 61.3%, $3 67.4%, $5 52.7%, $10 46.0%
Channel	Web 49.0% and social 47.7% vs email/mobile 44.0%
Gender	Female 56.4%, Other 54.7%, Male 43.2%, Unknown 11.6%
Age	Redemption rises with age: 18-24 37.2% up to 55-64 52.2%
Income	Higher income redeems more: 30-50K 35.7% up to 90-120K 61.3%
Tenure	2016 joiners best (59.0%); 2018 joiners lowest (28.0%)
Engagement	High 6,769 / Medium 3,727 / Low 6,498 / No offers 6; 12,774 customers completed at least one offer
Spending	Avg transaction $7.75 before first completion vs $15.63 after; non-completers $3.67

Recommendations

- Focus budget on discount offers and on low-to-mid rewards ($2–$5), which redeem better than $10 rewards.
- Use web and social channels in every campaign.
- Target lower-income, younger and male customers with easier offers to lift their redemption.
- Re-engage 2018 joiners and the 6,498 Low-engagement customers with simple, short offers.

- Collect gender, age and income at sign-up – the 2,175 customers without them redeem only 11.6%.